

机器学习 Lab 3 实验报告

PB22111599 杨映川

0.实验准备

本次实验在cpu上运行完成

1.GMM聚类

1.1 E-step

根据当前的参数计算每个样本属于每个聚簇的概率，获取 γ

```
1  def _e_step(self, X: np.ndarray) -> np.ndarray:
2      """
3          E-step: Compute the responsibilities.
4
5          Args:
6              - X: np.ndarray, shape (N, D), Data.
7
8          Returns:
9              - gamma: np.ndarray, shape (N, K), Responsibilities.
10     """
11     N, D = X.shape
12     gamma = np.zeros((N, self.n_components))
13     # print(gamma) # ok
14     # print(self.n_components) # 10
15     # self.covs = np.array([np.eye(data_dim) for _ in
16     range(n_components)])
17     # print(self.covs) # nan
18
19     # 1.1 E-step: Compute the responsibilities
20     # BEGIN_YOUR_CODE
21     # raise Exception("Not implemented yet")
22     for k in range(self.n_components):
23         try:
24             # Compute the inverse and determinant of the
25             covariance matrix
26             inv_cov = np.linalg.inv(self.covs[k])
27             det_cov = np.linalg.det(self.covs[k])
28
29             if det_cov <= 0:
30                 raise np.linalg.LinAlgError("Covariance matrix
31                 is not positive definite.")
32
33             # Compute the Gaussian probability density for all
34             samples for component k
```

```

31         gamma[:, k] = self.pi[k] * self._gaussian(X,
self.means[k], inv_cov, det_cov)
32     except np.linalg.LinAlgError:
33         # Handle singular covariance matrix by adding a
small value to the diagonal
34         print(f"Covariance matrix for component {k} is
singular. Adding regularization.")
35         self.covs[k] += 1e-6 * np.eye(D)
36         inv_cov = np.linalg.inv(self.covs[k])
37         det_cov = np.linalg.det(self.covs[k])
38         gamma[:, k] = self.pi[k] * self._gaussian(X,
self.means[k], inv_cov, det_cov)
39
40     # Normalize the responsibilities across all components for each
sample
41     gamma_sum = np.sum(gamma, axis=1, keepdims=True)
42     # To prevent division by zero
43     gamma_sum[gamma_sum == 0] = 1e-10
44     gamma /= gamma_sum
45     # END_YOUR_CODE
46     return gamma

```

1.2 M-step

使用最大似然估计更新模型参数

```

1  def _m_step(self, X: np.ndarray, gamma: np.ndarray):
2      """
3      M-step: Update the parameters.
4
5      Args:
6          - X: np.ndarray, shape (N, D), Data.
7          - gamma: np.ndarray, shape (N, K), Responsibilities.
8      """
9      N, D = X.shape
10     n_soft = np.sum(gamma, axis=0) # [K,]
11
12     # 1.2 M-step: Update the parameters
13     # BEGIN_YOUR_CODE
14     # raise Exception("Not implemented yet")
15     n_soft += 1e-10
16     self.pi = n_soft / N
17     self.means = gamma.T @ X / n_soft[:, None] # [K, D]
18
19     X_centered = X[:, np.newaxis, :] - self.means # [N, K, D]
20     gamma_expanded = gamma[:, :, np.newaxis] # [N, K, 1]
21     covs = np.einsum('nkd,nke->kde', X_centered, X_centered *
gamma_expanded) / n_soft[:, np.newaxis, np.newaxis]
22
23     covs += 1e-4 * np.eye(D)[None, :, :] # 防止奇异矩阵
24
25     for k in range(self.n_components):
26         if not np.all(np.linalg.eigvals(covs) > 0):
27             print("Covariance matrix is not positive definite")

```

```

28         covs += 1e-4 * np.eye(D) [None, :, :]
29
30         assert np.all(np.linalg.eigvals(covs) > 0), "Covariance
matrix is not positive definite"
31
32         self.covs = covs
33         # END_YOUR_CODE

```

2.PCA降维

2.1 计算主成分

完成fit方法，保留d个主成分

```

1  def fit(self, X: np.ndarray):
2      """
3      Fit the PCA model to the data.
4
5      Args:
6      - X: np.ndarray, shape (N, D), Data.
7      """
8      # 2.1 Compute the principal components
9      # BEGIN_YOUR_CODE
10     # raise Exception("Not implemented yet")
11     self.mean = np.mean(X, axis=0)
12     X_centered = X - self.mean
13     covariance_matrix = np.cov(X_centered, rowvar=False)
14     eigenvalues, eigenvectors =
np.linalg.eigh(covariance_matrix)
15
16     sorted_idx = np.argsort(eigenvalues)[::-1]
17     sorted_eigenvectors = eigenvectors[:, sorted_idx]
18
19     self.components = sorted_eigenvectors[:, :self.dim].T
20
21     # END_YOUR_CODE

```

2.2 从GMM中采样

实现了sample_from_gmm函数

```

1  def sample_from_gmm(gmm: GMM, pca: PCA, label: int, path:
Union[str, Path]):
2      """
3      Sample images from a Gaussian Mixture Model.
4
5      Args:
6      - gmm: GMM, Gaussian Mixture Model.
7      - pca: PCA, Principal Component Analysis.
8      - label: int, Cluster label.
9      """
10     # 5.1
11     # BEGIN_YOUR_CODE

```

```

12     mean = gmm.means[label]
13     cov = gmm.covs[label]
14
15     sample_low_dim = np.random.multivariate_normal(mean, cov)
16     sample_high_dim = pca.inverse_transform(sample_low_dim)
17     sample_scaled = np.clip(sample_high_dim, 0,
255).astype(np.uint8)
18     sample_image = sample_scaled.reshape(28, 28)
19     # raise Exception("Not implemented yet")
20     # END_YOUR_CODE
21
22     # Save an example image (you can change this part of the code if
you want)
23     # sample = Image.fromarray(sample[0], mode="L") # if sample
shape is (1, 28, 28)
24     # path = Path(path)
25     # sample.save(path / "gmm_sample.png")
26     img = Image.fromarray(sample_image, mode="L")
27     path = Path(path)
28     img.save(path / "gmm_sample.png")

```

3.调试过程

分别改变embedding_dim和max_iter的值使用PCA降维

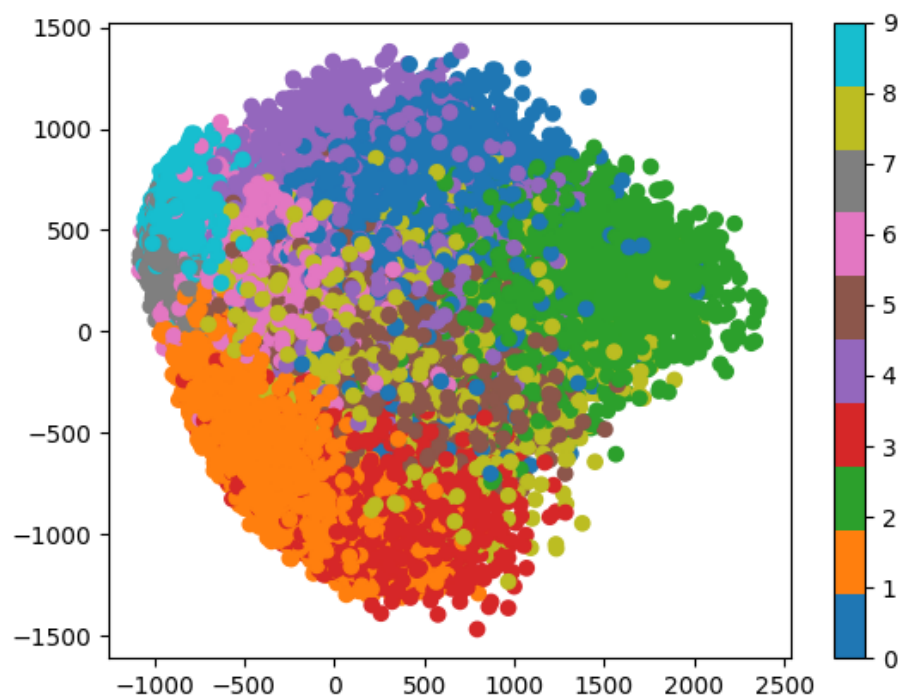
max_iter	50	100
embedding_dim	grade	grade
5	26.40	27.38
10	28.96	nan
15	29.27	26.76

4.最好实验结果

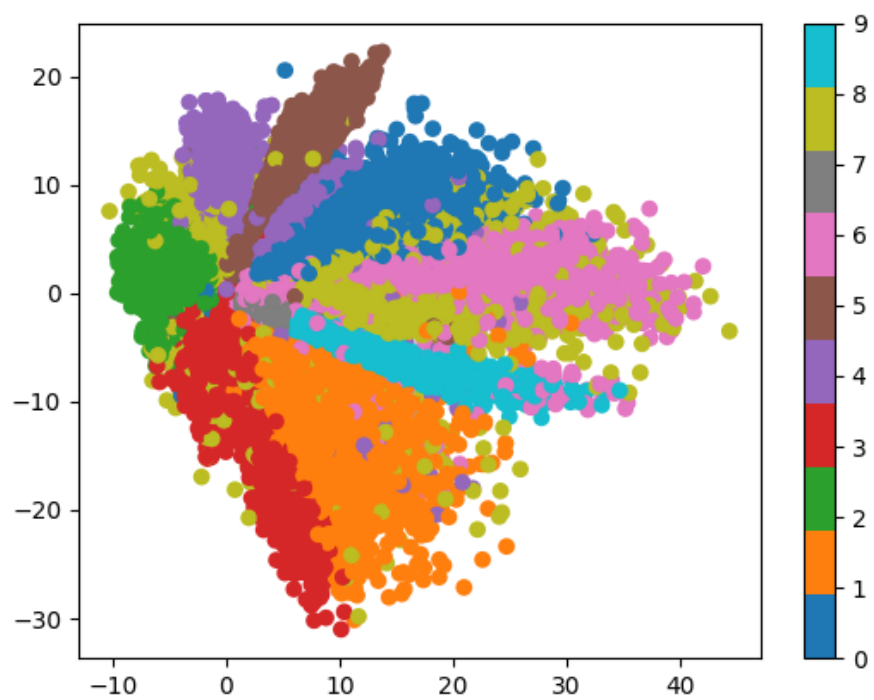
当max_iter=50, embedding_dim=15时有29.27的grade值

由visualization.py输出的按聚簇标签可视化的结果

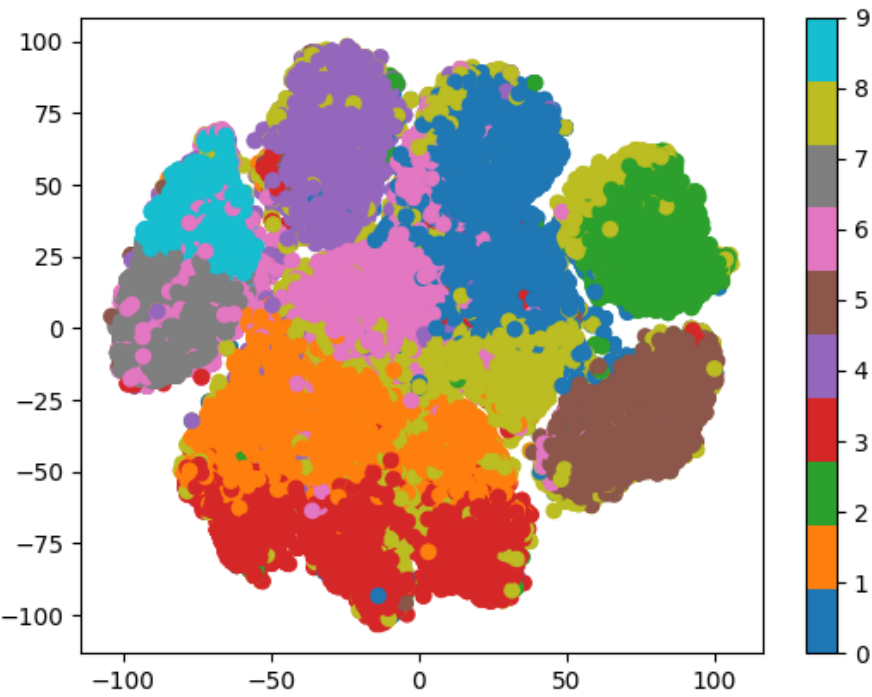
- pca



- ae



- tsne



GMM作为生成模型生成的图片

选择sample_index=6



5.回答问题

5.1 从训练速度,降维效率,灵活性(eg.是否适用于各种类型的数据),对数据分布的保持程度,可视化效果这几个方面比较PCA, tSNE, AutoEncoder这三种降维方法的优劣(你可以列一个表格)

	PCA	tSNE	AutoEncoder
训练速度	快	较慢，计算复杂度高	训练时间取决于网络复杂度，通常较慢
降维效率	高，适用于线性关系	中等，特别适用于高维复杂数据	高，能够捕捉复杂的非线性关系
灵活性	适用于各种数据类型	主要用于数值型数据	适用于各种数据类型
对数据分布的保持程度	保持全局结构，可能忽略局部结构	优秀，保持局部邻域结构，可能失真全局结构 良好，依赖于网络架构和训练质量	

	PCA	tSNE	AutoEncoder
可视化效果	中等，主要展示全局趋势	优秀，效果显著	良好，可以生成有意义的低维表示

5.2 从生成效率,生成质量,灵活性(eg.是否适用于各种类型的数据),是否可控(eg.生成指定类别的样本)这几个方面比较GMM和DDPM的优劣, (DDPM的原理参考书面作业第四题)

	GMM	DDPM
生成效率	高：涉及从各个高斯分布中采样，并进行加权混合，计算量较小	低：涉及多个去噪步骤（通常数百到数千步），计算量大
生成质量	中等：在复杂数据上表现有限	高：在复杂数据上表现很好
灵活性	中等：主要适用于连续、数值型数据	高：适用于多种数据类型，包括图像、音频、文本等
是否可控	有限：在复杂场景下难以实现精细控制	高:可以通过条件扩散模型（如条件DDPM）实现对生成样本的精确控制

6.反馈

- 用时：
 - 补全代码2h
 - 调试24h